Introduction to Microcontrollers

Facilitator Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

Introduction

This document is showing the list of experiments and procedures which will be conducted by students in this course

As Facilitator you should be in advance in the following topics:

Week 4: Understanding Analog Signals and Arduino

. Introduction to Analog Signals

Analog signals and digital signals are the two fundamental types of electrical signals used in electronics and communication. While both are crucial in different applications, understanding the differences between them and how to work with analog signals on an Arduino is vital in many projects.

---

2. Analog vs. Digital Signals

Digital Signals:

- **Digital signals** are binary signals that have only two states: HIGH or LOW (on or off). In Arduino, a digital signal is represented as either 5V (HIGH) or 0V (LOW).
- Digital signals are precise but limited in the type of data they can represent. They are ideal for simple on/off control, such as turning an LED on or off or reading a button press.

Analog Signals:

- **Analog signals** are continuous signals that can take any value within a range. Instead of just being on or off, an analog signal can represent a wide range of values.
- For example, a light sensor's output can vary from very low (in darkness) to very high (in bright light). Analog signals allow for more complex, nuanced control of devices like motors or LEDs.

Comparison:

Digital Signal	Analog Signal
0V or 5V (on or off)	Continuously varying values
Easy to work with	More precise but complex
Ideal for simple control	Used for sensors, motors, etc.

3. Using Analog Pins on the Arduino

Arduino comes with analog pins that are specially designed to read analog signals from sensors. These pins can measure a wide range of values, from 0 to 1023, corresponding to voltages from 0V to 5V.

Analog Pins:

- **Analog pins (A0 to A5)** are used to read analog signals. These pins take an input signal (such as from a sensor) and convert it into a digital value that can be used in your program.

Example of Analog Pin Usage:

You can connect a sensor like a potentiometer to an analog pin (e.g., A0) to measure changing values.

---

4. Analog Input: Reading Values from Sensors

Analog input allows Arduino to read values from sensors such as potentiometers, temperature sensors, and light sensors. These sensors output continuous signals that reflect real-world conditions, and Arduino can interpret these signals using its analog-to-digital converter (ADC).

Example: Reading from a Potentiometer:

A **potentiometer** is a variable resistor that changes its resistance based on how you rotate its knob. When connected to an analog pin, you can read its values and use them to control other devices.

```cpp
int sensorPin = A0; // Connect the potentiometer to A0
int sensorValue = 0; // Variable to store the sensor reading

void setup() {
  Serial.begin(9600); // Start serial communication to display values
}

void loop() {
  sensorValue = analogRead(sensorPin); // Read the value from the potentiometer
  Serial.println(sensorValue); // Print the value to the serial monitor
  delay(500); // Small delay before the next reading
}
```

In this example, we read the analog value from the potentiometer connected to **A0**. The value ranges from 0 to 1023, representing the voltage from 0V to 5V.

---

5. Analog Output: PWM (Pulse Width Modulation)

Although the Arduino's digital pins can only output HIGH or LOW signals, we can use a technique called **Pulse Width Modulation (PWM)** to simulate analog output. PWM rapidly turns the digital pin on and off, adjusting the ratio of on-to-off time to create the illusion of varying voltage levels.

PWM in Arduino:

- **PWM** allows you to control devices like motors, LEDs, and other components by adjusting their brightness or speed.
- On Arduino, PWM is available on digital pins marked with a ~ symbol (such as 3, 5, 6, 9, 10, and 11).

---

6. Controlling the Brightness of an LED Using PWM

Using PWM, we can control the brightness of an LED. By adjusting the duty cycle (the percentage of time the signal is HIGH), we can make the LED appear dim or bright.

Example: Controlling LED Brightness:

```cpp
int ledPin = 9; // LED connected to digital pin 9
int brightness = 0; // Variable to store the brightness level
int fadeAmount = 5; // How much to increase or decrease the brightness

void setup() {
  pinMode(ledPin, OUTPUT); // Set the LED pin as an output
}

void loop() {
  analogWrite(ledPin, brightness); // Set the brightness of the LED using PWM
  brightness = brightness + fadeAmount; // Change the brightness by the fade amount

  // Reverse the direction of the fading if it hits max or min brightness
  if (brightness <= 0 || brightness >= 255) {
    fadeAmount = -fadeAmount;
  }

  delay(30); // Wait for 30 milliseconds before the next loop
}
```

In this example:

- The LED is connected to pin 9 (a PWM pin), and we use analogWrite() to control the brightness.
- The brightness level is gradually increased and decreased in a fading effect, creating smooth transitions between dim and bright.

Summary:

- **Analog signals** are continuous and can represent a wide range of values, while **digital signals** are binary (on/off).
- Arduino uses **analog pins** to read varying sensor values and **PWM** to simulate analog output.
- **Analog input** allows Arduino to interface with sensors like potentiometers or temperature sensors, while **PWM** can control devices like LEDs or motors with variable intensity or speed.

By mastering analog signals and understanding how to use them on an Arduino, you can create projects that interact with the environment in more nuanced and dynamic ways.